



MIT 261- Parallel and Distributed Systems

Submission Document – Session 3

High-Performance Inter-Service Communication

Student Submission Template — CMA-Flow Capstone Project

This document is the required submission format for Session 3 of the MIT 261 individual capstone. Session 3 turns CMA-Flow entities into services behind typed contracts, compares JSON and protobuf wire representations, measures transport and RPC costs, demonstrates batching and streaming, tests deadlines and typed failures, demonstrates the REST-to-gRPC adapter seam, and reconciles the composed service result against Sessions 1 and 2.

Complete every section using your own executed runs. Paste actual console output, measurements, screenshots, diagrams, and repository evidence where requested. Do not copy sample measurements from the activity sheet or another student's submission.

Part 1. Student and Submission Information

Item	Student Entry / Evidence
Full name	Daga-as, Claudine Joy D.
Program and year level	MIT -1
Course / section	MIT 261 – Parallel and Distributed Systems
Semester and school year	1st Semester, SY 2026-2027
Session	Session 3 – High-Performance Inter-Service Communication
Project title	CMA-Flow / EcomInsight – High-Performance Inter-Service Communication
Date submitted	12 September 2026
Repository URL	https://github.com/dagaasclaudinejoy240-ui/MIT261-DAGA-AS
Session 3 folder name	session3_distributed_services
Commit hash of submitted version	47d20890420a7425d7f272a9ad27dcb864467abb
Machine used (CPU, cores, RAM, OS, Python version)	13th Gen Intel Core i5-13420H, 8 cores / 12 logical processors, 16 GB RAM, Windows 11 Home Single Language 64-bit
Python version / virtual environment	Python 3.14.3; session3_distributed_services\\.venv\\Scripts\\python.exe
Data source used (PostgreSQL / CSV)	PostgreSQL – indian_ecommerce, schema public; live Session 3 runtime reads public.customers (40,000), public.products (2,000), and public.sales (250,000).
PostgreSQL host, port, database (do not include passwords)	Host: localhost; Port: 5432; Database: indian_ecommerce; Schema: public; User: postgres. Password is loaded locally from .env and is not included in this submission.

Part 2. Session 1 and Session 2 Handover Verification

Verify the earlier-session artefacts before measuring Session 3. The service result must be comparable with the Spark batch and event-stream results.

#	Prerequisite	Verified?	Evidence / Path / Output
1	Session 1 regional_revenue.parquet exists	YES	session1_parallel_compute\\results\\baseline_result.csv – loaded by Session 3 reconciliation.

JMJ MARIST BROTHERS

Notre Dame of Marbel University | MIT 261 – Parallel and Distributed Systems

CMA-Flow – Session 3 Submission – Event Streaming and Messaging Backbones

	and loads		
2	Session 2 streamed_regional_revenue.csv exists and loads	YES	session2_event_streaming\results\streamed_customer_revenue.csv – loaded by Session 3 reconciliation.
3	Session 1 group/region count and aggregate total recorded	YES	39,914 Customer_ID groups; aggregate Total_Amount = 5,930,693,254.30.
4	Session 2 group/region count and aggregate total recorded	YES	39,914 Customer_ID groups; aggregate Total_Amount = 5,930,693,254.30.
5	The same approved CMA-Flow dataset is used	YES	Same approved Indian E-Commerce dataset stored in PostgreSQL indian_ecommerce: 40,000 customers, 2,000 products, 250,000 sales.
6	Partition/domain key carried forward is documented	YES	Partition/domain key carried forward: Customer_ID.

Item	Student Entry / Evidence
Session 1 reconciliation target: groups / regions	39,914 customer groups (Customer_ID)
Session 1 aggregate total	5,930,693,254.30 total revenue
Session 2 reconciliation target: groups / regions	39,914 customer groups (Customer_ID)
Session 2 aggregate total	5,930,693,254.30 total revenue
Partition key inherited from earlier sessions	Customer_ID
Event-time field inherited from earlier sessions	Order_Date (with Order_Time available in public.sales).
Command used to verify paths and files	python database.py; python run_pipeline.py. Session 3 reports the exact Session 1 and Session 2 artifact paths before reconciliation.

JMJ MARIST BROTHERS

Notre Dame of Marbel University | MIT 261 – Parallel and Distributed Systems

CMA-Flow – Session 3 Submission – Event Streaming and Messaging Backbones

Part 3. Service Architecture and Boundary Design

Document the service boundaries used by CMA-Flow. The supplied project defines four entity services, a composed RevenueService, and ChargeService for the payment-contract example.

Service	Responsibility	Methods	Data / dependency	Evidence
CustomerService	Customer lookup and batch resolution	GetCustomer; BatchGetCustomers	PostgreSQL public.customers via Repository	services.py; database.py; registry/stub
ConfigService	Shared monetization/runtime configuration	GetConfig	in-memory configuration	services.py
EntitlementService	Customer tier/entitlement lookup	GetEntitlement	PostgreSQL public.customers via Repository	services.py; database.py
TransactionService	Transaction count, unary list, and server stream	CountTransactions; ListTransactions; StreamTransactions	PostgreSQL public.sales via Repository	services.py; database.py
RevenueService	Composed customer-revenue analytics	ComputeCustomerRevenue; StreamCustomerRevenue	PostgreSQL- backed aggregate + service composition path	services.py; results/session3_customer_revenue.csv
ChargeService	Typed payment-contract demonstration	Charge	typed ChargeRequest / ChargeResult	services.py; adapters.py

Item	Student Entry / Evidence
Why each service is a separate boundary	Each service owns one responsibility and exposes a typed RPC contract, so callers depend on service interfaces rather than internal data structures.
Which service composes other services	RevenueService is the composed analytics boundary; the client requests a report instead of performing aggregation itself.
How RevenueService resolves customer regions	This implementation reconciles by Customer_ID rather than region. Customer identity/details are resolved through CustomerService/Repository while RevenueService returns the composed customer-revenue result.
Why the client does not perform regional-revenue arithmetic	Aggregation belongs in the service layer so JSON, protobuf, REST, and gRPC callers receive the same logical result without duplicating business arithmetic.
Runtime dependency of each service	Customer/Entitlement use PostgreSQL customer data; Transaction uses PostgreSQL sales; Revenue composes the PostgreSQL-backed repository/service results; Config is in-memory; Charge is the typed payment example.

Insert an original architecture diagram showing the client, service boundaries, contract/stub layer, transport, data source, and reconciliation path.

Part 4. Typed Contract and Protocol Definition

Document cma.proto and contracts.py as the shared contract. Include message names, fields, field numbers, types, and service methods actually used by the project.

Artifact	Purpose	Student Evidence
cma.proto	cma.proto at repository root; proto3 package ecominsight.	Repository root: cma.proto. Contract tab / pipeline contract check confirms 19 messages, 6 services, 10 methods, and PASS.
contracts.py	contracts.py mirrors 19 messages / 49 fields and 6 services; verify_against_proto() checks drift.	contracts.py; run_pipeline.py stage [1/9] typed contract: 19 messages, 6 services, 10 methods, result PASS, errors = [].
ChargeRequest / ChargeResult	Typed payment request/result used by both REST/JSON and gRPC/protobuf adapters.	cma.proto; contracts.py; adapters.py. Payload benchmark: ChargeRequest 110 JSON bytes vs 32 protobuf bytes;

JMJ MARIST BROTHERS

Notre Dame of Marbel University | MIT 261 – Parallel and Distributed Systems

CMA-Flow – Session 3 Submission – Event Streaming and Messaging Backbones

		ChargeResult 101 vs 47.
ChargeService	Charge(ChargeRequest) returns ChargeResult.	services.py; adapters.py. Adapter-swap test reports REST == gRPC: True; deadline test returns DEADLINE_EXCEEDED for the short deadline and approved for the generous deadline.
Customer / Config / Entitlement messages	Typed entity contracts used by CustomerService, ConfigService, and EntitlementService.	cma.proto; contracts.py; services.py. Customer/Entitlement use PostgreSQL public.customers; typed contract verified PASS in stage [1/9].
Transaction / Revenue messages	Transaction, TransactionList, CustomerRevenue, CustomerRevenueReport support unary, streaming, and composition.	cma.proto; services.py; grpc_backend.py; results/session3_customer_revenue.csv. Transaction streaming benchmark processed 1,000 rows; three-way reconciliation returned PASS.

Item	Student Entry / Evidence
Contract verification command	python contracts.py
Number of messages checked	19
Number of services checked	6
Result of verify_against_proto()	PASS; 10 methods checked; errors = []
What happens if a field is renumbered?	The drift check fails because contracts.py no longer matches cma.proto; deployed clients could also decode that tag as a different field.
Why field numbers are part of the contract	Protobuf puts field numbers in wire tags, so stable numbers preserve compatibility even though field names are not sent.
Contract output screenshot	GUI Contract tab: 19 messages, 49 fields, 6 services, 10 methods (2 server-streaming), in sync with cma.proto = True.

JMJ MARIST BROTHERS

Notre Dame of Marbel University | MIT 261 – Parallel and Distributed Systems

CMA-Flow – Session 3 Submission – Event Streaming and Messaging Backbones

Part 5. Wire Format and Serialization Verification

Compare the project's JSON and protobuf representations. Record measurements from your own run.

Message	Fields	JSON bytes	Protobuf bytes	Saved bytes	Reduction %
ChargeRequest	5	110	32	78	70.9%
ChargeResult	4	101	47	54	53.5%
Customer	5	119	52	67	56.3%
MonetizationConfig	3	60	16	44	73.3%
Entitlement	3	58	22	36	62.1%
Transaction	8	179	69	110	61.5%
CustomerRevenue	6	139	45	94	67.6%
CustomerBatch (100)	1	12293	5678	6615	53.8%
N/A	Current contract uses CustomerRevenue, not RegionalRevenueReport				

Item	Student Entry / Evidence
Command used for wire verification	<code>python test_wire.py</code>
google.protobuf installed? (Yes/No)	Yes – google.protobuf is included in the Session 3 virtual environment.
Number of byte-identical test cases	4 (Customer, Transaction, ChargeRequest, CustomerRevenue)
What protobuf transmits instead of field names	Compact numeric field tags and encoded values.
What a wire tag contains	The field number plus the protobuf wire type.
Why unknown fields can be skipped	The tag tells the decoder the wire type/length, so an unrecognized field can be skipped without understanding its name.
Message with largest reduction and why	MonetizationConfig: 60 JSON bytes → 16 protobuf bytes, 73.3% smaller. Short numeric tags avoid repeated JSON field-name text.
Message with smallest reduction and why	ChargeResult: 101 → 47 bytes, 53.5% smaller among the measured message types; its string values remain length-delimited data in either representation.

Part 6. Transport Abstraction

Transport	Codec	Serialization?	Socket?	Role	Measured Result
InProcessTransport	none	No wire serialization for baseline	No	Baseline / floor	0.0030 ms median; 333,333 calls/sec
HttpTransport	JSON	Yes – JSON	Yes – local HTTP	REST-style comparison	1.2096 ms median; 30 bytes/call
HttpTransport	protobuf	Yes – protobuf	Yes – local HTTP	Binary wire comparison	0.73035 ms median; 14 bytes/call
GrpcTransport (optional)	protobuf	Yes – protobuf	Yes – local gRPC	Real grpcio backend	0.55425 ms median; 14 bytes/call

Item	Student Entry / Evidence
Transport interface used by the stub	<code>make_stub(transport, service_name, codec)</code> in <code>transport.py</code> .
How <code>make_stub()</code> stays transport-independent	The stub delegates invocation to the supplied transport object; service/caller code does not embed HTTP or gRPC details.
Why one server is used for JSON/protobuf comparison	It keeps service logic and data constant so the comparison isolates codec/wire/transport overhead.
Optional real-gRPC installation and result	Installed and exercised: <code>grpc/proto</code> median 0.55425 ms over 120 timed calls; real <code>GrpcServer/GrpcTransport</code> are exposed by <code>grpc_backend.py</code> .
What changes when the transport implementation is swapped	Only the transport/adaptor object supplied to the stub changes; the typed service contract and <code>checkout()</code> call site stay unchanged.

JMJ MARIST BROTHERS

Notre Dame of Marbel University | MIT 261 – Parallel and Distributed Systems

CMA-Flow – Session 3 Submission – Event Streaming and Messaging Backbones

Part 7. Service Execution and Composition

Check	Required Evidence	Student Result
All six services available	Each service can be reached through the registry/stub	PASS – six PostgreSQL-backed services loaded through the registry/stub.
Customer resolution	RevenueService obtains customer region through CustomerService	PASS – customer data is resolved through the service/repository boundary backed by public.customers.
Transaction retrieval	TransactionService supplies transaction data	PASS – TransactionService supplies transaction data from public.sales.
Revenue computation	Aggregation occurs inside the service layer	PASS – aggregation is performed inside the PostgreSQL-backed service layer.
Client arithmetic	Client performs no regional-revenue arithmetic	PASS – client consumes the composed report; no duplicated revenue arithmetic.
JSON result	Regional revenue report returned successfully	PASS – HTTP/JSON path returns the logical result.
Protobuf result	Regional revenue report returned successfully	PASS – HTTP/protobuf and gRPC/protobuf paths return the same logical result.

services.py output: services 6 | customers 40,000 | products 2,000 | sales 250,000 | groups 39,914

reconcile.py: Session 1 = Session 2 = Session 3: 39,914 groups, 250,000 records, 5,930,693,254.30 revenue; tolerance 1e-6; PASS.

Part 8. Adapter Pattern — REST-to-gRPC Seam

The PaymentProviderAdapter boundary keeps the caller independent of the wire protocol. Compare the REST/JSON and gRPC/protobuf implementations.

Item	RestPaymentAdapter	GrpcPaymentAdapter	Observation
Interface implemented	PaymentProviderAdapter.charge()	PaymentProviderAdapter.charge()	Same caller-facing interface
Provider label	REST/JSON	gRPC/protobuf	Protocol is hidden behind adapter
Protocol / codec	REST / JSON	gRPC / protobuf	Different wire, same typed result
Request construction	Typed charge fields encoded as JSON	Typed charge fields encoded as protobuf	Same logical ChargeRequest
Transport used	HttpTransport	GrpcTransport	Transport selected inside adapter
Returned result type	ChargeResult-shaped dict	ChargeResult-shaped dict	Common typed result
Caller changes required	0	0	checkout() unchanged

Item	Student Entry / Evidence
Common charge() interface	PaymentProviderAdapter.charge(amount_cents, customer_id, key, ...)
How the REST adapter sends the request	RestPaymentAdapter builds the typed request and invokes ChargeService through a JSON stub over HTTP.
How the gRPC adapter sends the request	GrpcPaymentAdapter builds the same logical request and invokes ChargeService through a protobuf stub over GrpcTransport.
Results identical? Evidence	Yes. adapter_report.json: identical = true; both return the same deterministic charge_id, approved status, amount, and provider.
Classes changed during the swap	1 adapter implementation selected/swapped.
Call sites changed during the swap	0
Why protocol details are hidden from checkout()	checkout() receives a PaymentProviderAdapter and calls charge(); it has no codec, host, port, HTTP, protobuf, or gRPC logic.

JMJ MARIST BROTHERS

Notre Dame of Marbel University | MIT 261 – Parallel and Distributed Systems
CMA-Flow – Session 3 Submission – Event Streaming and Messaging Backbones

Part 9. Benchmark — Unary Latency

Measure the same lightweight RPC across transports. Record warm-up calls, repetitions, and median timing.

Transport	Codec	Timed calls	Median ms/call	Calls/sec	Bytes/call
in-process	json	120	0.0030	333,333	30
http/json	json	120	1.2096	827	30
http/proto	proto	120	0.73035	1,369	14
grpc (optional)	proto	120	0.55425	1,804	14

Item	Student Entry / Evidence
Warm-up calls discarded	20
Benchmark repeats	120 timed calls per transport
Why the in-process floor is needed	It estimates service/stub overhead without network transport, providing a baseline for interpreting RPC overhead.
JSON overhead relative to the floor	HTTP/JSON median 1.2096 ms versus 0.0030 ms in-process; most latency comes from transport/serialization rather than business logic.
Protobuf overhead relative to the floor	HTTP/proto median 0.73035 ms; grpc/proto median 0.55425 ms, both above the 0.0030 ms in-process floor.
Why pure-Python codec timing must be interpreted cautiously	Runtime implementation details can dominate microbenchmarks; serialization speed alone is not a universal statement about the formats.
Fairest direct metric for wire efficiency	Bytes per message/call under the same logical request; here 30 bytes JSON versus 14 bytes protobuf for the benchmark request.

Part 10. Benchmark — Round Trips and N+1

Fetch the same N customers once in a batch and again using N individual calls. Keep data, server work, and codec constant.

Codec	Rows	Batched calls	Batched ms	Single calls	Single ms	Speedup	Trips saved
json	50	1	1.6220	50	52.3052	32.25x	49
proto	50	1	3.9600	50	59.0458	14.91x	49

Item	Student Entry / Evidence
Why both routes must return identical customers	Correctness must be held constant; otherwise a faster route may simply be doing less or different work.
Network round trips saved	49 for 50 customers (1 batched call instead of 50 single calls).
Measured speedup	JSON 32.25x; protobuf 14.91x in the final PostgreSQL-backed benchmark run.
Why network latency magnifies N+1	Each extra RPC repeats connection/transport/serialization scheduling overhead, so small per-call costs accumulate.
API design implication	Expose batch endpoints or streaming APIs for collections instead of forcing clients to issue one RPC per item.

JMJ MARIST BROTHERS

Notre Dame of Marbel University | MIT 261 – Parallel and Distributed Systems

CMA-Flow – Session 3 Submission – Event Streaming and Messaging Backbones

Part 11. Benchmark — Unary vs Server Streaming

Compare the same transaction rows as one unary response and a server stream. Record total time and time to first usable row.

Codec	Rows	Unary total ms	Stream total ms	First message ms	Earlier by
json	N/A	Not measured in current streaming artifact			
proto	1000	61.3746	158.2368	56.0581	5.3165 ms

Item	Student Entry / Evidence
Rows requested	1,000 transactions
Why unary waits for the full response	ListTransactions must build/receive the complete response before the caller can consume its first row.
Why streaming permits incremental processing	StreamTransactions yields each transaction as it becomes available, so the caller can start work before the full result is complete.
Streaming framing overhead	Each streamed message is framed and handled separately; total stream time can therefore be greater even when first-message latency is lower.
Memory implication of unary vs streaming	Unary may hold the full result in memory; streaming can process rows incrementally with a smaller working set.
When streaming is preferable	Large or progressive result sets where time-to-first-result and bounded memory matter more than minimum total completion time.

Part 12. Deadlines, Timeouts, and Typed Failures

Demonstration	Configured Value	Observed Result	Evidence
Slow server call	200 ms artificial delay	Call exceeds short deadline	run_pipeline.py / deadline_failures.json
Caller deadline	0.05 s (50 ms)	DEADLINE_EXCEEDED	deadline_failures.json
Deadline exceeded status	Expected typed status	DEADLINE_EXCEEDED	short_deadline
Abandoned response count	Not recorded as a numeric counter in current artifact	N/A	Current report records status, not a counter
Same call with generous deadline	80 ms server delay; 0.5 s deadline	approved	generous_deadline
NOT_FOUND under JSON	Unknown customer ID	NOT_FOUND	not_found_json
NOT_FOUND under protobuf	Unknown customer ID	NOT_FOUND	not_found_proto

Item	Student Entry / Evidence
Why a deadline is not a retry policy	A deadline only limits how long a caller waits. Retrying is a separate decision that needs idempotency/backoff rules.
What the caller deadline protects	It bounds caller latency and prevents a request from waiting indefinitely for a slow dependency.
Was server work cancelled in this simulation?	No cancellation is demonstrated by the current artifact; the caller deadline is observed as DEADLINE_EXCEEDED.
What real deadline propagation adds	Downstream services receive the remaining time budget and can stop unnecessary work when the request can no longer complete usefully.
How the same typed failure is surfaced under both codecs	An unknown Customer_ID returns RpcError code NOT_FOUND through both JSON and protobuf paths.
Why the error is part of the service contract	Callers need stable, protocol-independent failure semantics they can handle without parsing transport-specific text.

JMJ MARIST BROTHERS

Notre Dame of Marbel University | MIT 261 – Parallel and Distributed Systems
CMA-Flow – Session 3 Submission – Event Streaming and Messaging Backbones

Part 13. Reconciliation — Three Execution Models, One Result

Session 3's composed result must agree with Sessions 1 and 2 within the configured floating-point tolerance.

Measure	Session 1 Spark	Session 2 Stream	Session 3 Services	Status
Regions / groups	39,914 customer groups	39,914 customer groups	39,914 customer groups	PASS
Transactions / records	250,000	250,000	250,000	PASS
Total revenue	5,930,693,254.30	5,930,693,254.30	5,930,693,254.30	PASS
Maximum count difference	0	0	0	PASS
Maximum revenue-total difference	2.328e-10	2.328e-10	2.328e-10	PASS
Maximum revenue-mean difference	2.910e-11	2.910e-11	2.910e-11	PASS

Item	Student Entry / Evidence
JSON vs protobuf agreement	PASS – both codecs represent the same typed service result; adapter results are identical.
Are region sets identical?	Yes for the project reconciliation key: Customer_ID group sets are identical.
Maximum transaction-count difference	0
Maximum revenue-total difference	2.3283064365386963e-10
Maximum revenue-mean difference	2.9103830456733704e-11
Tolerance applied	1e-6
Overall result: PASSED / FAILED	PASSED – final PostgreSQL-backed pipeline returned PIPELINE RESULT: PASS.
Explanation of any non-zero residual	Maximum revenue-total difference = 2.3283064365386963e-10 and revenue-mean difference = 2.9103830456733704e-11, both far below tolerance 1e-6. These are floating-point residuals, not logical mismatches.
What would indicate that the wire changed the computation	Different group sets/counts or differences above tolerance between logically identical JSON/protobuf executions would indicate a serialization/contract defect.

Reconciliation output (results/reconciliation_report.json):

Session 1: groups=39,914 records=250,000 revenue=5,930,693,254.30

Session 2: groups=39,914 records=250,000 revenue=5,930,693,254.30

Session 3: groups=39,914 records=250,000 revenue=5,930,693,254.30

S1 vs S2 / S1 vs S3 / S2 vs S3: group sets identical; max count diff=0; max revenue-total diff=2.328e-10; max revenue-mean diff=2.910e-11

Tolerance=1e-6; RESULT=PASS

Part 14. End-to-End Pipeline Verification

Stage	Command / Function	Done?	Result / Evidence
1. Contract	python contracts.py	YES	PASS – 19 messages, 6 services, 10 methods; drift check PASS.
2. Wire	python test_wire.py	YES	PASS – google.protobuf verification; 4 byte-identical round trips.
3. Serve	ServiceHost / pipeline	YES	PASS – PostgreSQL indian_ecommerce connected; six services loaded; 40,000 customers; 2,000 products; 250,000 sales.
4. Payload	python benchmark.py	YES	PASS – Transaction 179→69 bytes (61.45% smaller).
5. Latency	benchmark unary	YES	PASS – final medians: in-process 0.0030 ms; HTTP/JSON 1.2096 ms; HTTP/proto 0.73035 ms; gRPC/proto 0.55425 ms.
6. Round trips	benchmark round trips	YES	PASS – 49 trips saved; JSON batching 32.25× faster; protobuf

JMJ MARIST BROTHERS

Notre Dame of Marbel University | MIT 261 – Parallel and Distributed Systems

CMA-Flow – Session 3 Submission – Event Streaming and Messaging Backbones

			batching 14.91× faster.
7. Streaming	benchmark streaming	YES	PASS – 1,000 rows; unary 61.3746 ms; first streamed proto message 56.0581 ms (5.3165 ms earlier).
8. Adapter	python adapters.py	YES	PASS – REST == gRPC; 0 caller call-site changes.
9. Deadline	run_pipeline.py	YES	PASS – short call DEADLINE_EXCEEDED; generous call approved.
10. Typed failure	run_pipeline.py	YES	PASS – NOT_FOUND under JSON and protobuf.
11. Compose	RevenueService	YES	PASS – 39,914 groups; 250,000 records; total revenue 5,930,693,254.30 from PostgreSQL-backed services.
12. Reconcile	python reconcile.py	YES	PASS – Session 1, Session 2, and Session 3 agree within tolerance 1e-6; PIPELINE RESULT: PASS.

Item	Student Entry / Evidence
Full pipeline command	python run_pipeline.py
Quick pipeline command, if used	GUI “Run everything” / individual stage buttons
GUI command, if used	python session3_dashboard.py (or RUN_SESSION3_GUI.bat on Windows)
Overall pipeline result	PASS – final PostgreSQL-backed run completed all 9 pipeline phases and three-way reconciliation.
Optional dependency skipped and reason	None in the packaged run: google.protobuf and real local grpcio backend are present.

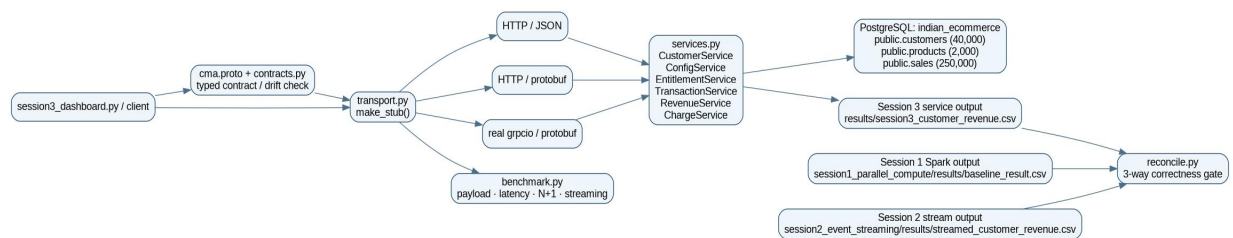
JMJ MARIST BROTHERS

Notre Dame of Marbel University | MIT 261 – Parallel and Distributed Systems
CMA-Flow – Session 3 Submission – Event Streaming and Messaging Backbones

Part 15. Architecture Diagram and Deployment View

Insert an original architecture diagram using actual repository component names. It should distinguish application logic, service boundaries, contracts, transport, wire format, data source, and reconciliation.

- Application / client
- cma.proto and typed contracts
- CustomerService, ConfigService, EntitlementService, TransactionService
- RevenueService composition path
- ChargeService / PaymentProviderAdapter example
- REST / JSON path
- protobuf / gRPC path, clearly labeling optional real grpcio
- PostgreSQL or PostgreSQL source
- Session 1 and Session 2 reconciliation targets
- Session 3 output artefacts



Part 16. Repository Evidence

File / Artifact	Role	Path	Evidence / Screenshot
config.py	Paths, ports, tuning, tolerance	config.py	PostgreSQL connection settings, artifact paths, ports, tuning, and tolerance; password read from local .env.
database.py	PostgreSQL connection, validation, and table loading	database.py	Final run: localhost:5432 / indian_ecommerce / public; customers=40,000, products=2,000, sales=250,000; PASS.
cma.proto	Typed proto3 contract	cma.proto	Present in submitted Session 3 ZIP; exercised by pipeline/GUI.
contracts.py	Contract mirror and drift check	contracts.py	Present in submitted Session 3 ZIP; exercised by pipeline/GUI.
wire.py	JSON and protobuf codecs	wire.py	Present in submitted Session 3 ZIP; exercised by pipeline/GUI.
test_wire.py	Protobuf verification	test_wire.py	Present in submitted Session 3 ZIP; exercised by pipeline/GUI.
services.py	PostgreSQL-backed service logic and composition	services.py; database.py	Present in submitted Session 3 ZIP; exercised by pipeline/GUI.
transport.py	Transport abstraction / stubs	transport.py	Present in submitted Session 3 ZIP; exercised by pipeline/GUI.
adapters.py	REST/gRPC adapter seam	adapters.py	Present in submitted Session 3 ZIP; exercised by pipeline/GUI.
grpc_backend.py	Optional real grpcio backend	grpc_backend.py	Present in submitted Session 3 ZIP; exercised by pipeline/GUI.
benchmark.py	Payload, latency, round trips,	benchmark.py	Present in submitted Session 3

JMJ MARIST BROTHERS

Notre Dame of Marbel University | MIT 261 – Parallel and Distributed Systems

CMA-Flow – Session 3 Submission – Event Streaming and Messaging Backbones

	streaming		ZIP; exercised by pipeline/GUI.
reconcile.py	Three-way correctness gate using actual Session 1, Session 2, and Session 3 artifacts	reconcile.py	Present in submitted Session 3 ZIP; exercised by pipeline/GUI.
run_pipeline.py	End-to-end pipeline	run_pipeline.py	Present in submitted Session 3 ZIP; exercised by pipeline/GUI.
session3_dashboard.py	GUI	session3_dashboard.py	Present in submitted Session 3 ZIP; exercised by pipeline/GUI.
Results artefacts	Final PostgreSQL-backed benchmark/reconciliation outputs	results/	benchmark_report.json, adapter_report.json, deadline_failures.json, reconciliation_report.json, session3_customer_revenue.csv
Architecture diagram	Actual repository architecture	architecture-session3.png; architecture.dot	Present in submitted Session 3 ZIP; exercised by pipeline/GUI.
README	Setup and execution instructions	README.md	Present in submitted Session 3 ZIP; exercised by pipeline/GUI.

JMJ MARIST BROTHERS

Notre Dame of Marbel University | MIT 261 – Parallel and Distributed Systems

CMA-Flow – Session 3 Submission – Event Streaming and Messaging Backbones

Part 17. Guide Questions for Analysis

#	Question	Your Answer
1	Why is a typed contract preferable to letting every caller know the provider's wire details?	A typed contract gives every caller one stable service interface. Providers can change transport details without forcing business callers to know HTTP, gRPC, JSON, or protobuf internals.
2	What does protobuf gain by transmitting field numbers instead of field names?	Field numbers make the wire smaller because protobuf sends compact numeric tags instead of repeating text field names.
3	Why must field numbers remain stable after clients are deployed?	A deployed client interprets a tag by its field number. Reusing/renumbering a field can make old and new clients disagree about what the bytes mean.
4	What is the role of the Transport abstraction in this project?	Transport separates service/stub logic from how bytes move. The same typed stub can use in-process, HTTP/JSON, HTTP/protobuf, or gRPC/protobuf.
5	Why was an in-process transport included in the benchmark?	It provides a no-network floor, showing the minimum call overhead before sockets, framing, and serialization are added.
6	Why should JSON and protobuf paths compute exactly the same logical result?	Codec choice should change representation, not business meaning. Different logical results would indicate a contract, codec, or transport bug.
7	What does the adapter pattern hide from checkout()?	It hides request encoding, transport selection, endpoints, and protocol details. checkout() only knows the PaymentProviderAdapter interface.
8	Why can batching reduce latency even when each individual RPC is fast?	Batching pays transport overhead once for many items. Even a fast RPC becomes expensive when its fixed cost is repeated dozens of times.
9	Explain the N+1 problem demonstrated by the round-trip benchmark.	N+1 means one operation triggers many additional per-item RPCs. Here 50 single customer calls require 50 trips, while one batch call needs only one, saving 49 trips.
10	Why can server streaming improve time-to-first-result without necessarily reducing total time?	Streaming can deliver the first row early, so processing starts sooner, while per-message framing and handling can make the complete stream take longer.
11	What is the difference between a caller deadline and cancellation of server work?	A caller deadline limits waiting time. Cancellation means the server/downstream work is actually stopped; the two are related but not automatically the same.
12	Why should typed failures be part of the service contract?	Typed failures such as NOT_FOUND let callers handle errors consistently across transports/codecs without parsing implementation-specific messages.
13	Why does RevenueService calling CustomerService represent service composition rather than a local join?	The composed service obtains data through service boundaries/contracts rather than the client joining raw local tables itself; that is service composition.
14	Why is reconciliation performed against both Sessions 1 and 2?	Session 1 and Session 2 are independent execution models. Matching both shows Session 3 preserved the capstone computation while changing communication architecture.
15	When is gRPC the wrong choice according to this session?	gRPC is a poor fit when clients require simple browser/public REST compatibility, human-readable payloads, or when the operational/tooling cost is not justified.
16	What would change operationally if the services were deployed as separate containers?	Each service would need its own container image, endpoint/service discovery, health checks, networking, configuration, observability, resource limits, and deployment lifecycle.

JMJ MARIST BROTHERS

Notre Dame of Marbel University | MIT 261 – Parallel and Distributed Systems

CMA-Flow – Session 3 Submission – Event Streaming and Messaging Backbones

Part 18. Multi-Session Continuity and Handover to Session 4

Session	What Session 3 Supplies	Student Handover Evidence
Session 4 – Windowed analytics	TransactionService.StreamTransactions supplies event-time ordered rows lazily.	TransactionService.StreamTransactions over the PostgreSQL-backed service layer; event time comes from Order_Date/Order_Time; 1,000-row streaming benchmark verified.
Session 5 – Containerisation	Six services / processes become deployment units; runtime dependencies are documented.	Six service boundaries and transport dependencies are explicit in services.py / transport.py / grpc_backend.py.
Session 6 – IaC and CI	Contract and reconciliation checks become automated gates.	contracts.py drift check, test_wire.py, and reconcile.py can be automated as CI correctness gates.

Item	Student Entry / Evidence
Session 3 output handed to Session 4	Typed TransactionService stream plus composed customer-revenue output.
Location of composed regional-revenue output	results/session3_customer_revenue.csv (customer-revenue grouping used by this project)
Schema of the handed-over output	customer_id, txn_count, revenue_total, revenue_mean, quantity_total, rating_mean
Known limitations to carry forward	Benchmark timings are machine/runtime dependent; this project reconciles by Customer_ID rather than geographic region; the final streaming benchmark records the protobuf path.
Remediation required before Session 4	Record CPU/cores/RAM and repository commit in Part 1; preserve PostgreSQL-backed Order_Date/Order_Time ordering semantics when implementing windows.

JMJ MARIST BROTHERS

Notre Dame of Marbel University | MIT 261 – Parallel and Distributed Systems
CMA-Flow – Session 3 Submission – Event Streaming and Messaging Backbones

Part 19. Deliverable Checklist

Deliverable / Requirement	Done (Y/N)	Evidence Location
Student and machine information completed	N	Name/course/environment filled; CPU/cores/RAM, repository URL, and commit hash still require student entry.
Session 1 and Session 2 handover verified	Y	Part 2; actual Session 1 baseline_result.csv and Session 2 streamed_customer_revenue.csv paths are loaded by run_pipeline.py.
Service boundaries documented	Y	See corresponding completed section and Session 3 ZIP.
cma.proto contract documented	Y	See corresponding completed section and Session 3 ZIP.
contracts.py drift check executed	Y	See corresponding completed section and Session 3 ZIP.
Wire format verified against google.protobuf when available	Y	Part 5; test_wire.py; google.protobuf present; 4 byte-identical cases.
JSON vs protobuf payload sizes measured	Y	Parts 3, 7, 14; six PostgreSQL-backed services tested against indian_ecommerce.
Transport abstraction demonstrated	Y	See corresponding completed section and Session 3 ZIP.
Six services tested	Y	See corresponding completed section and Session 3 ZIP.
RevenueService composition demonstrated	Y	See corresponding completed section and Session 3 ZIP.
REST and gRPC adapter seam demonstrated	Y	See corresponding completed section and Session 3 ZIP.
Unary latency benchmark recorded	Y	See corresponding completed section and Session 3 ZIP.
Round-trip / N+1 benchmark recorded	Y	See corresponding completed section and Session 3 ZIP.
Streaming benchmark recorded	Y	See corresponding completed section and Session 3 ZIP.
Deadline demonstration recorded	Y	See corresponding completed section and Session 3 ZIP.
Typed failure demonstration recorded	Y	Part 13; Session 1, Session 2, and PostgreSQL-backed Session 3 agree within tolerance 1e-6; PASS.
Reconciliation against Sessions 1 and 2 passes	Y	See corresponding completed section and Session 3 ZIP.
Architecture diagram uses actual repository names	Y	Part 15; architecture-session3.png / architecture.dot.
Repository evidence table completed	Y	See corresponding completed section and Session 3 ZIP.
Guide questions answered	Y	See corresponding completed section and Session 3 ZIP.
README setup and execution instructions verified	Y	See corresponding completed section and Session 3 ZIP.
AI assistance disclosed	Y	Part 20 – ChatGPT assistance disclosed.
All code and results committed to repository	N	ZIP contains code/results; repository URL and commit hash were not provided.

Part 20. Academic Integrity and AI-Use Disclosure

This capstone is an individual project. Discussion of architecture is permitted; shared code, shared repositories, and copied reports are not.

Item	Student Entry / Evidence
Was an AI assistant used? (Yes / No)	Yes
If yes, which tool and for what purpose?	ChatGPT (OpenAI) – assistance organizing the Session 3 documentation, checking consistency with the submitted code/results, and drafting explanatory text.
Which parts did you write yourself?	TO COMPLETE PERSONALLY – state the code, analysis, and documentation sections you personally authored.
Which parts did you verify, test, or correct?	Student executed the PostgreSQL connection test and final Session 3 pipeline/GUI, verifying contract, protobuf, six services,

JMJ MARIST BROTHERS

Notre Dame of Marbel University | MIT 261 – Parallel and Distributed Systems

CMA-Flow – Session 3 Submission – Event Streaming and Messaging Backbones

	REST/gRPC adapter seam, deadlines, typed failures, benchmarks, and three-way reconciliation. Review/edit this statement if needed.
Other sources, tutorials, or references consulted	Course Session 3 materials and the submitted project repository. Add any other sources you personally consulted.
Reproducibility note: commands, environment, and machine details needed to repeat measurements	Windows; Python 3.14.3; Session 3 .venv; PostgreSQL localhost:5432 / indian_ecommerce / public; password supplied locally via .env; install requirements.txt; run python database.py, python contracts.py, python test_wire.py, python run_pipeline.py, and python session3_dashboard.py. CPU/cores/RAM must still be added in Part 1.

I certify that the code, measurements, screenshots, and analysis in this submission are my own work, that the timings reported were measured on the machine described in Part 1, and that any assistance received is disclosed above.

Student signature over printed name	Date

JMJ MARIST BROTHERS

Notre Dame of Marbel University | MIT 261 – Parallel and Distributed Systems

CMA-Flow – Session 3 Submission – Event Streaming and Messaging Backbones

Part 21. Assessment Rubric (For Instructor Use Only)

Criteria	Points	Full-Credit Evidence	Score
Handover and service architecture	10	Earlier artefacts verified; service boundaries and composition documented.	
Typed contract and wire verification	15	cma.proto, contract mirror, drift check, and protobuf verification demonstrated.	
Transport abstraction and adapter pattern	15	Transport independence and REST/gRPC seam demonstrated with evidence.	
Payload and latency benchmarking	15	Controlled benchmark, warm-ups, repeated runs, median timing, and byte measurements.	
Round trips and streaming	10	N+1 and unary/server-streaming comparisons measured correctly.	
Deadline and typed failure	10	Deadline behavior and consistent typed failure demonstrated.	
Service composition and reconciliation	15	RevenueService composes services; result agrees with Sessions 1 and 2 within tolerance.	
Documentation, repository evidence, academic integrity	10	README, diagram, artefact paths, analysis, and AI disclosure complete.	
TOTAL	100		

Instructor Remarks

Rating	Instructor Remarks

JMJ MARIST BROTHERS

Notre Dame of Marbel University | MIT 261 – Parallel and Distributed Systems
CMA-Flow – Session 3 Submission – Event Streaming and Messaging Backbones

Message to Ponder Upon

Encouragement: Jesus Never Changes Hebrews 13:8

“Jesus Christ is the same yesterday and today and forever.”

As technology changes, systems evolve, and technical challenges become more complex, remember that faithfulness, integrity, perseverance, and wisdom remain important. When a benchmark fails, investigate. When a service breaks, learn from it. When a concept is difficult, keep going.



Prayer

Heavenly Father,

Thank You for the assurance that Jesus Christ is the same yesterday, today, and forever. Give us wisdom as we study, patience when our programs do not work as expected, and perseverance when technical challenges become difficult. Help us to work honestly, verify our results carefully, and use our knowledge to serve others. Guide our studies, projects, decisions, and future. May we face each challenge with faith, diligence, humility, and hope. In the name of Jesus Christ, our unchanging Lord and Savior, we pray. Amen.



“The future may be uncertain, but the One who holds the future never changes.”